
Лабораторная работа 2 по floatam

Выполнил:
Ромашков Семён

МФТИ
2025 год

Пункт 0: Преобразование unsigned int в двоичное представление

Код программы:

```
1 #include <iostream>
2
3 int main(){
4     for (unsigned int n = 1; n < 1000000000; n *= 10){
5         auto mask = 1u << 31;
6         std::cout << n << ' ';
7         for(;mask > 0;mask = mask >> 1){
8             std::cout << ((n & mask) != 0);
9         }
10        std::cout << std::endl;
11    }
12    return 0;
13 }
```

Вывод: Получили двоичное представление разных чисел. Так их видит компьютер.

```
D:\Romashkov_mipt\Labs_inf\Lab_2_inf_Romashko...
1 00000000000000000000000000000001
10 000000000000000000000000000001010
100 0000000000000000000000000001100100
1000 000000000000000000000000111101000
10000 000000000000000000000010011100010000
100000 0000000000000000000011000011010100000
1000000 00000000000001110100001001000000
10000000 0000000010011000100101101000000
100000000 00000101111101011110000100000000

Process returned 0 (0x0)   execution time : 0.108 s
Press any key to continue.
```

Рис. 1: Двоичное представление unsigned int числа

Пункт 1: Преобразование float в двоичное представление

Теоретическая справка: Формат IEEE 754 для одинарной точности (32 бита) включает:

- Биты 0-22: мантисса (23 бита)
- Биты 23-30: экспонента (8 бит)
- Бит 31: знак (1 бит)

Код программы:

```
1 #include <iostream>
2
3 union FloatUnsigned {
4     unsigned Unsigned;
5     float Float;
6 }flu;
7 int main(){
8     std::cin >> flu.Float;
9     unsigned int n = flu.Unsigned;
10    auto mask = 1u << 31;
11    for(;mask > 0;mask = mask >> 1){
12        std::cout << ((n & mask) != 0);
13        if (mask == (1u << 31) || mask == (1u << 23)) {
14            std::cout << " ";
15        }
16    }
17    std::cout << std::endl;
18    // IEEE 754
19    unsigned int sign = (n >> 31) & 1;
20    unsigned int exponent = (n >> 23) & 0xFF;
21    unsigned int mantissa = n & 0x7FFFFFFF;
22    std::cout << sign << ' ';
23    mask = 1u << 7;
24    for (;mask > 0;mask = mask >> 1){
25        std::cout << ((exponent & mask) != 0);
26    }
27    std::cout << ' ';
28    mask = 1u << 22;
29    for (;mask > 0;mask = mask >> 1){
30        std::cout << ((mantissa & mask) != 0);
31    }
32 }
```

Вывод: Рассмотрели 2 разных варианта преобразования float в двоичное представление.

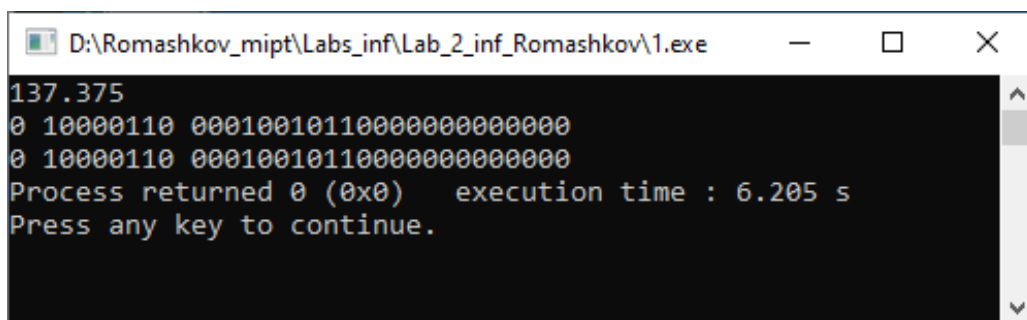


Рис. 2: Двоичное представление float числа согласно IEEE 754

Пункт 2: Исследование переполнения мантиссы

Код программы:

```
1 #include <iostream>
2
3 union FloatUnsigned {
4     unsigned Unsigned;
5     float Float;
6 }flu;
7
8 int main(){
9     std::cin >> flu.Float;
10    unsigned int n = flu.Unsigned;
11    auto mask = 1u << 31;
12    for(;mask > 0;mask = mask >> 1){
13        std::cout << ((n & mask) != 0);
14        if (mask == (1u << 31) || mask == (1u << 23)) {
15            std::cout << " ";
16        }
17    }
18    std::cout << std::endl;
19
20    // IEEE 754
21    unsigned int sign = (n >> 31) & 1;
22    unsigned int exponent = (n >> 23) & 0xFF;
23    unsigned int mantissa = n & 0x7FFFFFFF;
24    std::cout << sign << ' ';
25    mask = 1u << 7;
26    for (;mask > 0;mask = mask >> 1){
27        std::cout << ((exponent & mask) != 0);
28    }
29    std::cout << ' ';
30    mask = 1u << 22;
31    for (;mask > 0;mask = mask >> 1){
32        std::cout << ((mantissa & mask) != 0);
33    }
34 }
```

Вывод: Как видно, пока степень десятки невелика, ее двоичное разложение полностью влезает в мантиссу. Потом места уже не хватает, нули заканчиваются, и хвост двоичного разложения банально обрезается. Никаких предупреждений или ошибок при этом не будет. Просто вместо степени десятки у вас будет какое-то число, которое где-то рядом, но точного равенства уже не будет. Это связано с дискретностью диапазона float – в нем представимы далеко не все вещественные числа.

```
D:\Romashkov_mipt\Labs_inf\Lab_2_inf_Romashkov\2.exe
10.00 0 10000010 010000000000000000000000
100.00 0 10000101 100100000000000000000000
1000.00 0 10001000 111101000000000000000000
10000.00 0 10001100 001110001000000000000000
100000.00 0 10001111 100001101010000000000000
1000000.00 0 10010010 111010000100100000000000
10000000.00 0 10010110 001100010010110100000000
100000000.00 0 10011001 011111010111100001000000
1000000000.00 0 10011100 110111001101011001010000
10000000000.00 0 10100000 00101010000001011111001
99999997952.00 0 10100011 01110100100001110110111
999999995904.00 0 10100110 11010001101010010100101
9999999827968.00 0 10101010 00100011000010011100111
100000000376832.00 0 10101101 01101011110011000100001
999999986991104.00 0 10110000 11000110101111110101001
100000000272564224.00 0 10110100 00011100001101111001010
99999998430674944.00 0 10110111 01100011010001010111100
999999984306749440.00 0 10111010 10111100000101101101011

Process returned 0 (0x0)   execution time : 0.116 s
Press any key to continue.
```

Рис. 3: Явление переполнения мантиссы при операциях с float

Пункт 4: Численное интегрирование методом прямоугольников

Код программы:

```
1 #include <iostream>
2 #include <fstream>
3 #include <iomanip>
4 #include <cmath>
5
6 int main(){
7     std::ofstream f_float("4_float.csv", std::ios::out);
8     std::ofstream f_double("4_double.csv", std::ios::out);
9     for (auto n = 1; n < 100000; n++){
10         float h1 = M_PI / n;
11         float intergal1 = 0;
12         for (float x = 0.0; x <= M_PI; x += h1){
13             intergal1 += h1 * std::sin(x);
14         }
15         f_float << std::fixed;
16         f_float << std::setprecision(12);
17         f_float << n << ' ' << intergal1 << std::endl;
18
19         double h2 = M_PI / n;
20         double intergal2 = 0;
21         for (double x = 0.0; x <= M_PI; x += h2){
22             intergal2 += h2 * std::sin(x);
23         }
24         f_double << std::fixed;
25         f_double << std::setprecision(12);
26         f_double << n << ' ' << intergal2 << std::endl;
27     }
28     return 0;
29
30 }
```

Вывод: В файл записывались числа с 12 знаками после запятой, чтобы более точно исследовать нужные нам зависимости.

По графику видим, что сходимость метода прямоугольников на разбиении $[0, \pi]$ на 10 точек уже даёт относительную погрешность где-то 0.5%.

Сравнивая методы прямоугольников, где использовались float и double, видно, что сходимость метода прямоугольников примерно одинаковая. Различия видны в зависимости ошибки от количества точек: где-то до 300-400 точек разбиения нет отличий, а потом ошибка метода, основанного на float, начинает вести себя непредсказуемо. А вот ошибка метода, основанного на double, плавно падает с количеством точек.

